1. How long did you spend working on the problem? What did you find to be the most difficult part?
    a. It took me approximately 4.5 to 6.5 hours to implement this code.
    b. 2 to 3 hours to implement core logic, 1.5 to 2 for adding tests and edge cases, and lastly 1 to 1.5 for documentation and refactoring.
    c. Most difficult part: The trickiest part was handling concurrent access to shared data structures. The current implementation has race conditions.
2. How would you modify your data model or code to account for more kinds of metrics?
    a. I would update the device object to add device status (online, offline).
    b. A backlog size/queue, which will represent the number of heartbeats/stats remaining in a queue to be received. (in other words, unregistered heartbeats/stats).
    c. An object to store the average time to recovery. This will help in identifying the time needed for a device to restart/recover.
    d. Battery level, if the device is wireless. This will help in identifying the device's average battery life and energy consumption.
3. Discuss your solution's runtime complexity
    a. Memory complexity: O(m*n)
       m = number of devices in csv
       n = list of heartbeats/uploadtimes per device

       Time complexity
    b. RegisterHeartbeat - O(1), worst case O(n)
       O(1) = map lookup for device from device list
       O(1) = append for heartbeat to heartbeats array, worst case it will be O(n)
    c. RegisterStats - O(1), worst case O(n)
       O(1) = map lookup for device from device list
       O(1) = append for upload time to UploadTime list, worst case it will be O(n)
    d. GetStats - O(n)
       O(n) = iterate through all upload times to calculate average upload time
    e. LoadDevices - O(n)
       O(n) = number of devices in CSV